

This system is an Emergency incident response and dispatch management system called TaskForge. Managing emergency operations involves composite hierarchies, changing responder states, emergency incidents with added situational cautions and traversal for dispatch. Dispatch managers manage the composite hierarchy's interaction with emergencies. Responders have concrete operational state transitions available, dispatched, operating and recovering. Emergencies dynamically allow requirements to be added for example hazardous, structural or medical situations that change the capability. Dispatch algorithms inspect emergency units across composite trees using traversal strategies.

Composite Pattern:

Component(ResponseUnit): declares the uniform interface for all response entities.

Leaf(Responder): individual field responder having an assigned role and capability.

Composite(ResponseTeam): represents aggregated units storing a vector<ResponseUnit*> members collection

Iterator Pattern

Iterator(Iterator): Abstract base that declares cursor traversal operations (hasNext() and next()), maintaining snapshot buffer(vector<ResponseUnit*> toVisit)

Concrete Iterator(RollCallIterator): Depth-first traversal that traverses the composite tree using recursion, flattening all leaf responders into an ordered roll call snapshot.

Concrete Iterator2(DispatchIterator): Filtered selection traversal that checks emergency issues. It tries to match whole teams that fulfill requirements.

Aggregate (ResponseUnit): Defines an interface for creating an Iterator object

ConcreteAggregate(ResponseTeam, Responder) Implements the factory methods createRollCallIterator() and createDispatchIterator(Emergency*).

State Pattern

Context(ResponseUnit): Maintains an owning reference pointer State* state and delegates operations using performDuty() to current concrete state

State(State): Abstract base defining lifecycle interface, performDuty() and clone()

ConcreteState(Available, Dispatched, Operating, Recovering): Available team is idle at standbyLocation ready for dispatch, Dispatched unit is on route to destination, Operating unit is engaged in executing the assigned emergency, Recovering unit is temporarily unavailable for a duration rejecting queueing requests/

Decorator Pattern

Component(Emergency) Abstract interface defining incident properties: getLocation(), getDescription(), getRequiredResponders(), and getRequiredCapabilities().

Concrete Component(EmergencyIncident): incident carrying base location, description required baseline responder count and main capability requirements

Decorator(EmergencyCondition): Abstract wrapper maintaining an owned Emergency* emergency wrapped pointer sending calls to the wrappee.

Concrete Decorator(MedicalCondition, HazmatCondition, StructuralCondition): adds dynamic constraints, descriptions and injects required capacities into get RequiredCapabilities()

Our architecture makes sure there is a clear ownership hierarchy and clean polymorphic cycles through all the subsystems mainly to prevent memory leaks and unwanted behaviours. Every abstract base class defines a virtual destructor, so that strict deletion responsibilities are held. In composite hierarchy, ResponseTeam acts as owner of its child components, therefore when

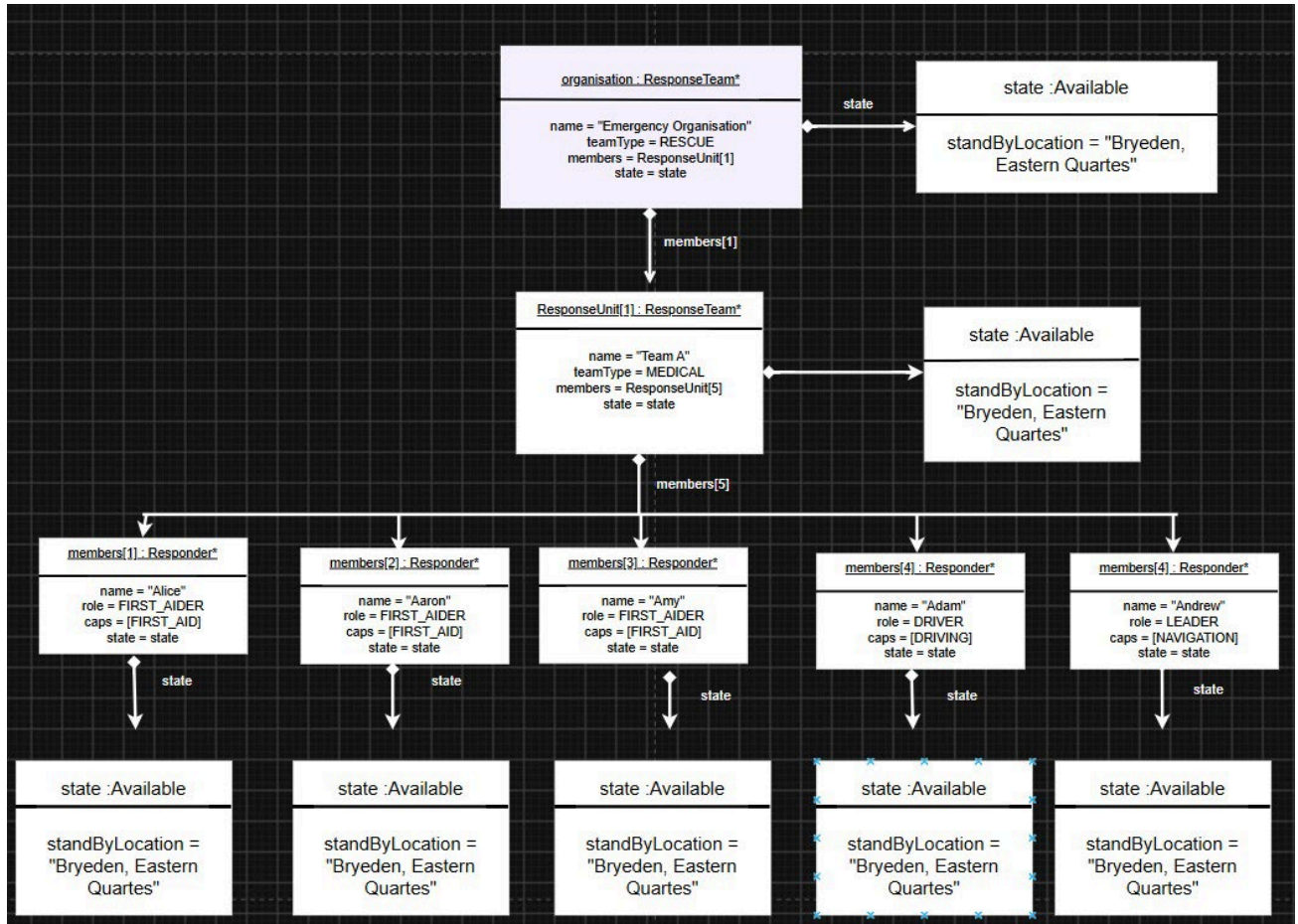
deletion occurs all nested units are deleted due to recursion. EmergencyCondition also takes ownership of its wrapped Emergency pointer making sure that dynamically stacked decorator chains clearly unwind and deallocate from the outermost layer. To avoid the double-free hazards that shared pointers could create, to avoid this state transitions use the Prototype pattern, when a composite team transitions, it calls clone() on its target state, providing every child unit with its own State instance. We also protect encapsulation by preventing client code like dispatch manager from accessing STL collections directly.

Task 3

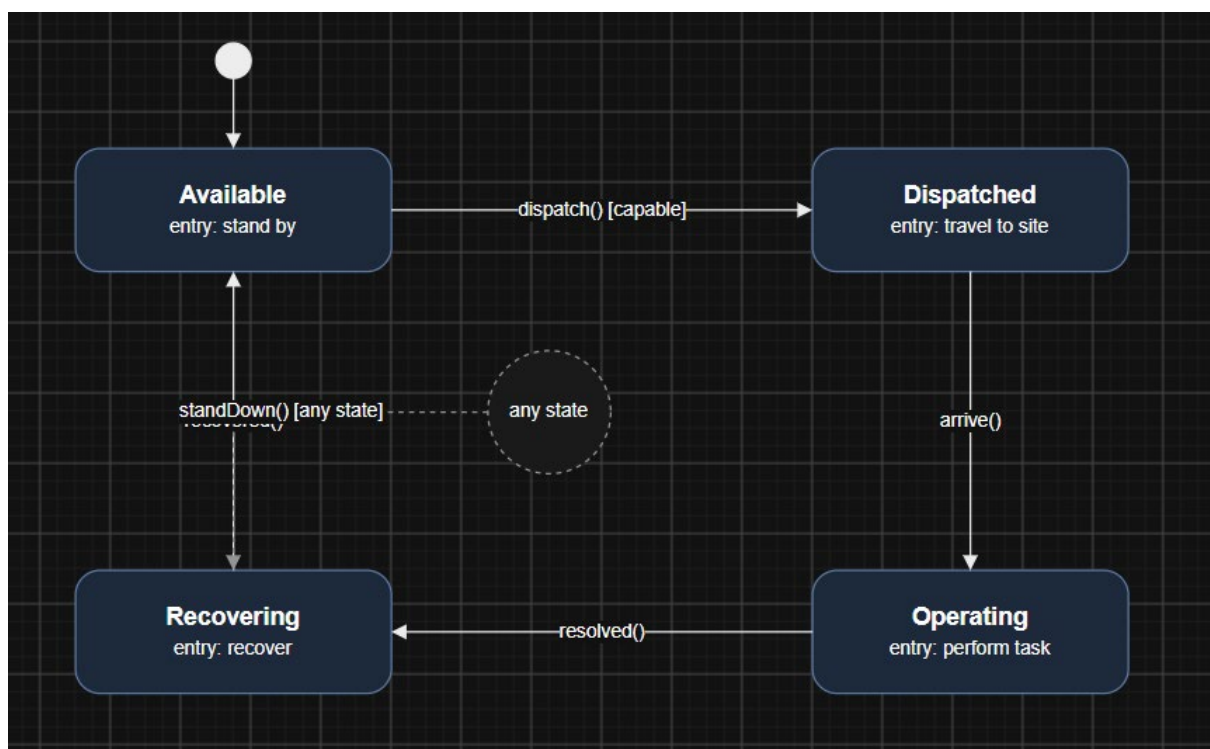
To make sure there is stability in changing runtime environments our system uses a snapshot traversal policy. When an iterator is created it captures the current composite tree into an internal vector<ResponseUnit*> toVisit buffer, tracking the progress with an position. If an update occurs, a new iterator will need to be created to view the updated tree to prevent unexpected behaviour while traversing and operating on a changing tree. A scenario, incoming emergencies invoke DispatchIterator to find qualified teams or assemble matching responders. Dispatched units cycle through states through setState() changing from Available, Dispatched, Operating to Recovering(10) back to Available. While in Recovering, isAvailable returns false which rejects any new dispatches and incidents with queuing into pendingEmergencies. DispatchManager restructures the hierarchy, and extracts the participating responders from their teams and calculates their dominant capability and packages all of them into a ResponseTeam and attaches it back to the root with the name “Emergency <emergencyCount> Team”.

Task 4

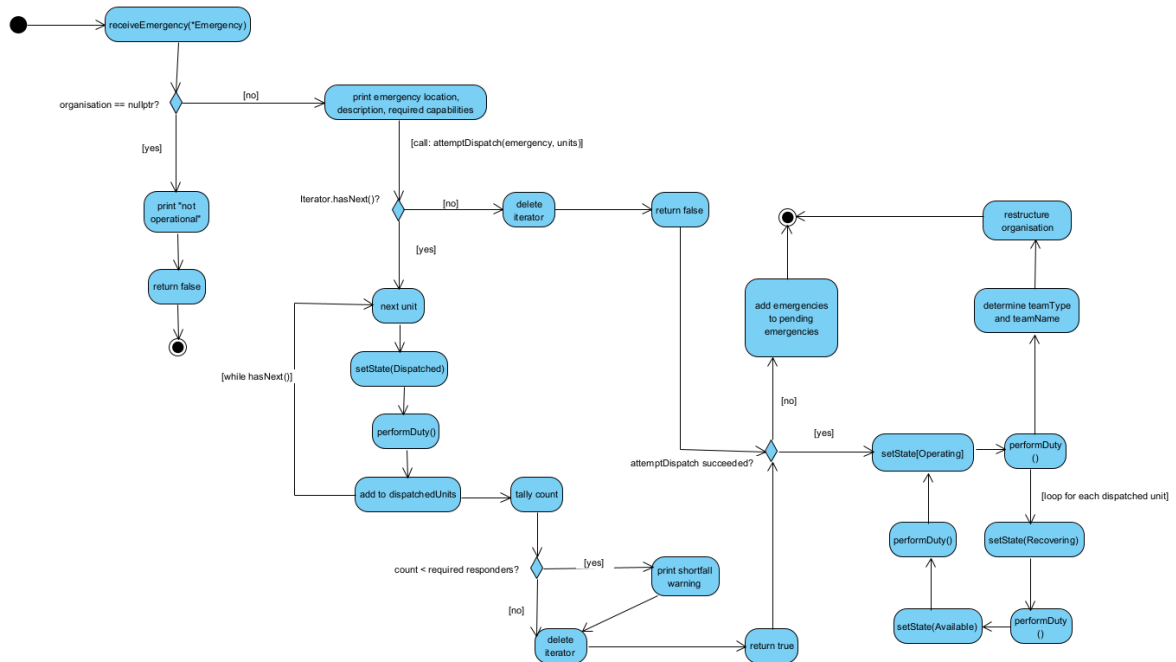
Object Diagram: An Organisation



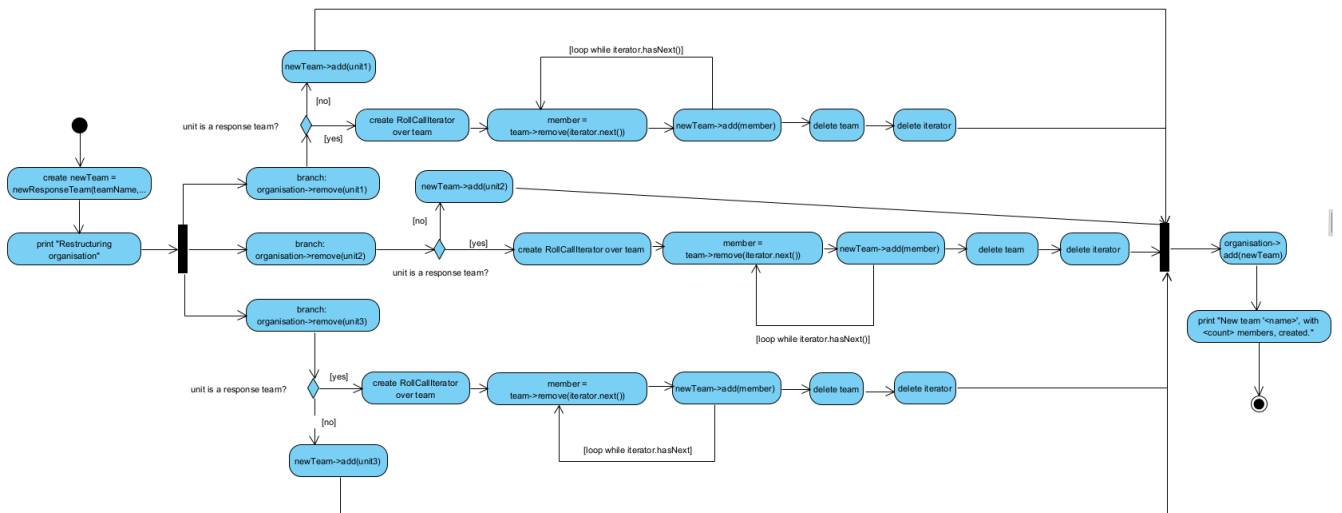
State Diagram: ResponseUnit's Lifecycle



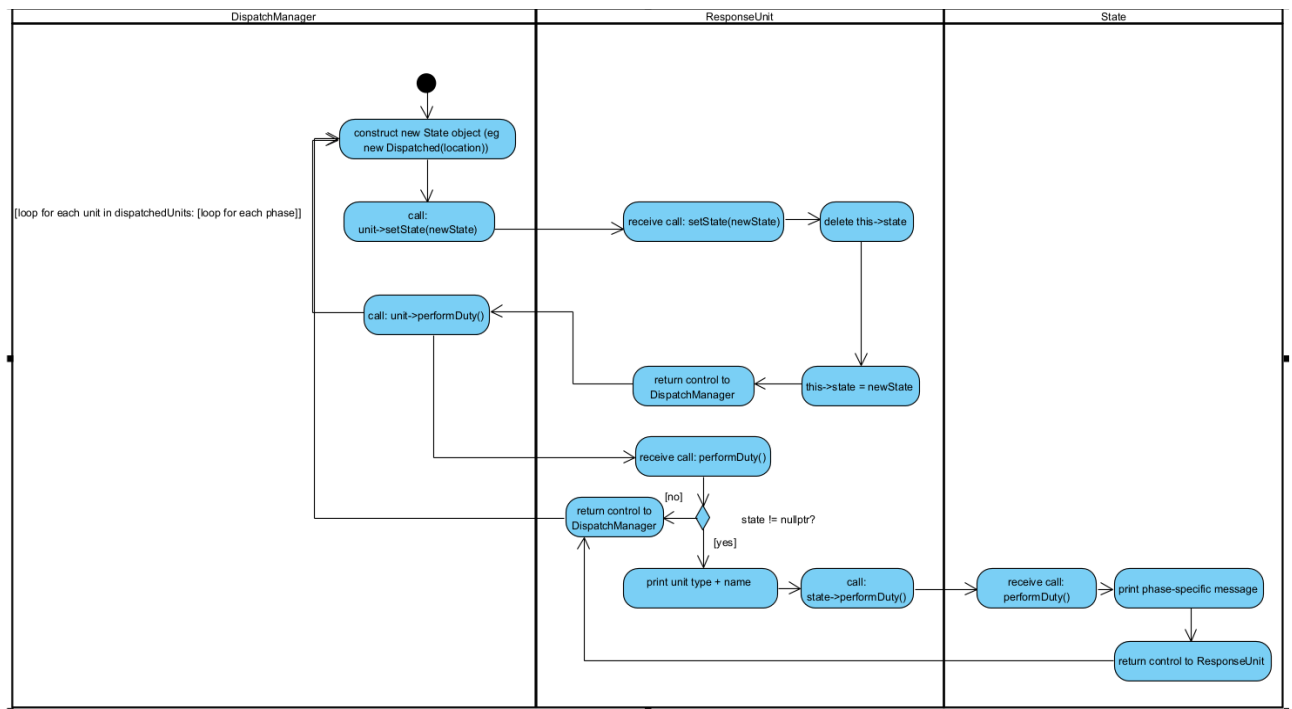
Activity Diagram 1: Receiving and dispatching an emergency



Activity Diagram 2: Post-Dispatch Reorganization



Activity Diagram 3: Unit lifecycle during emergency response



Task 5 — Debugging and Memory Investigation

5.1 GDB Decorator Investigation

GDB was used to investigate how the Decorator pattern builds the required capabilities of a stacked emergency. Breakpoints were placed on `getRequiredCapabilities()` in the different emergency condition decorators and the calls were stepped through one layer at a time. The inspected vectors showed the capability requirements being passed from the underlying `EmergencyIncident` through each decorator, with each decorator adding its own required capability before returning the resulting vector. This demonstrated the dynamic behaviour of the Decorator hierarchy and confirmed that the final vector contains the combined requirements of all applied conditions

```
Breakpoint 1, MedicalCondition::getRequiredCapabilities (this=0x55555583ed0) at MedicalCondition.cpp:8
8   std::vector<Capability> MedicalCondition::getRequiredCapabilities() {
(gdb) next
9   std::vector<Capability> capabilities = emergency->getRequiredCapabilities();
(gdb) step
StructuralCondition::getRequiredCapabilities (this=0x55555584070) at StructuralCondition.cpp:8
8   std::vector<Capability> StructuralCondition::getRequiredCapabilities() {
(gdb) next
9   std::vector<Capability> capabilities = emergency->getRequiredCapabilities();
(gdb) step
EmergencyIncident::getRequiredCapabilities (this=0x55555584090) at EmergencyIncident.cpp:32
32   return baseCapabilities;
(gdb) print baseCapabilities
$1 = std::vector of length 2, capacity 2 = {Capability::RESCUE, Capability::FIRST_AID}
(gdb) next
33   }
(gdb) finish
Run till exit from #0 EmergencyIncident::getRequiredCapabilities (this=0x55555584090) at EmergencyIncident.cpp:33
StructuralCondition::getRequiredCapabilities (this=0x55555584070) at StructuralCondition.cpp:10
10   capabilities.push_back(Capability::RESCUE);
Value returned is $2 = std::vector of length 2, capacity 2 = {Capability::RESCUE, Capability::FIRST_AID}
(gdb) print capabilities
$3 = std::vector of length 2, capacity 2 = {Capability::RESCUE, Capability::FIRST_AID}
(gdb) next
11   return capabilities;
(gdb) print capabilities
$4 = std::vector of length 3, capacity 4 = {Capability::RESCUE, Capability::FIRST_AID, Capability::RESCUE}
(gdb) finish
Run till exit from #0 StructuralCondition::getRequiredCapabilities (this=0x55555584070) at StructuralCondition.cpp:11
MedicalCondition::getRequiredCapabilities (this=0x55555583ed0) at MedicalCondition.cpp:10
10   capabilities.push_back(Capability::FIRST_AID);
Value returned is $5 = std::vector of length 3, capacity 4 = {Capability::RESCUE, Capability::FIRST_AID, Capability::RESCUE}
(gdb) print capabilities
$6 = std::vector of length 3, capacity 4 = {Capability::RESCUE, Capability::FIRST_AID, Capability::RESCUE}
(gdb) next
11   return capabilities;
(gdb) print capabilities
$7 = std::vector of length 4, capacity 4 = {Capability::RESCUE, Capability::FIRST_AID, Capability::RESCUE,
    Capability::FIRST_AID}
(gdb) |
```

5.2 GDB Bug Investigation

During testing, all emergencies were unexpectedly not being resolved even though all requirements should have been met. GDB was used to investigate the dispatch process by setting a breakpoint in the `DispatchIterator::hasNext()` and printing the value of `DispatchIterator::position` and the size of the `toVisit()` array. Inspection of the iterator's state revealed that position had not been initialised in the constructor, causing the iterator to begin with an invalid position. The constructor was corrected to initialise position to 0, after which the dispatch behaviour was correct.

```
PROCESSING EMERGENCIES...

=====EMERGENCY=====
New emergency received by organisation 'Emergency Organisation'.

Location of emergency: Central Hospital
Description: Large medical incident
Capabilities required: First-Aid Driving Navigation

Breakpoint 2, DispatchIterator::hasNext (this=0x555555584dd0) at DispatchIterator.cpp:98
98      return position < toVisit.size();
(gdb) print position
$1 = 3275369764234862650
(gdb) print toVisit.size()
$2 = 1
(gdb) █
```

5.3 Valgrind Investigation

Valgrind was used on the final application with full leak checking enabled. The final run reported no definitely lost memory and no errors originating from the project code, confirming that dynamically allocated memory was properly managed.

```
=====EMERGENCY=====
New emergency received by organisation 'Emergency Organisation'.

Location of emergency: Industrial Area
Description: Chemical spill
Capabilities required: Hazmat Driving Hazmat

Emergency unresolved. Adding to pending emergencies...
=====EMERGENCY=====
New emergency received by organisation 'Emergency Organisation'.

Location of emergency: Residential Area
Description: Collapsed structure with injured civilians
Capabilities required: Rescue First-Aid Rescue First-Aid

Emergency unresolved. Adding to pending emergencies...
=====

NULL MANAGER TEST...
The organisation is not currently operational.
==24132==
==24132== HEAP SUMMARY:
==24132==    in use at exit: 0 bytes in 0 blocks
==24132== total heap usage: 847 allocs, 847 frees, 110,269 bytes allocated
==24132==
==24132== All heap blocks were freed -- no leaks are possible
==24132==
==24132== For lists of detected and suppressed errors, rerun with: -s
==24132== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Task 6 — Git and Collaboration Reflection

The team began by collaboratively designing the system and dividing the initial UML work. The Composite and State hierarchies were developed separately from the Iterator and Emergency hierarchies. During implementation, the work was divided into the Composite and Emergency hierarchies, the Decorator hierarchy, and the main program, DispatcherManager, State and Iterators.

The initial UML designs were committed directly to the main branch. Once coding began, separate branches were used for development. The implementations were integrated by pulling the other members' branches into the testing branch (merges **db3095e** and **e97dd87**), allowing the complete system to be tested together before the final merge (merge **4c8351a**). The remaining diagrams and documentation were divided between team members, while GitHub issues were created during the final phase to track tasks that still needed to be finalised.

Repository URL: <https://github.com/KatNikKing/COS-214-Prac-4.git>